Generalization and Ensemble Learning

Lecture 28

CS 4880

Kirk Duran

Why Can a Perfect Decision Tree Be a Bad Model?

Memorizing old outcomes and predicting future outcomes are different achievements.

- Last lecture, an unrestricted tree could split until nearly every training example was classified correctly.
- A rule like "this exact camera pattern means blocked" may describe *one recording*, not a useful property of corridors.
- The robot must succeed on **new delivery runs**, new buildings, and new conditions.

Key idea

The goal is **generalization**: reliable predictions on observations not used to fit or choose the model.

Two different reports

Training accuracy: 100%

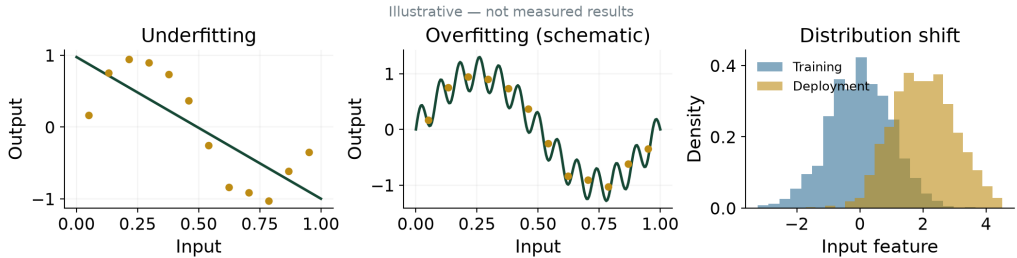
New-run accuracy: 67%

A high first number does not guarantee a high second one.

Hypothetical example, not a measured result.

Three Ways Predictions Fail

Ask whether the model is too simple, too specific, or facing a changed world.



- **Underfitting:** a depth-one tree misses important interactions.
- **Overfitting:** a deep tree turns unusual training examples into fragile rules.
- **Distribution shift:** the deployment environment differs from training (e.g., rain, lighting, or a new campus building).

Train, Validate, Test: Each Set Has One Job

Keep model fitting, model selection, and final reporting separate.

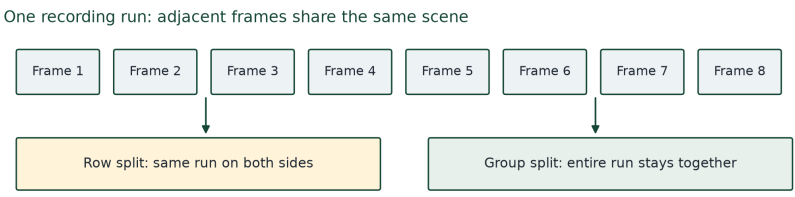
Partition	What we do with it	Question it answers
Training	Fit weights, splits, and leaf predictions.	What relationships can the model learn?
Validation	Choose depth, forest size, and threshold.	Which candidate looks best on unseen cases?
Test	Evaluate the frozen selected model *once*.	How well did the chosen procedure transfer?

Key idea

Set hyperparameters on validation, learn splits on training, and report the final test only after choosing the model.

Data Leakage: Different Frames, Same Experience

A random row split is not independent if rows record nearly the same event.

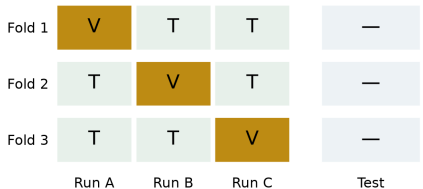


Near-duplicate recording frames can leak across a frame-level split.

- Adjacent frames share nearly the same corridor, lighting, and obstacles.
- Sharing a run between training and test gives an *overly optimistic* score.
- Keep delivery runs/sessions intact. To predict future runs, hold out later time periods; random grouped splits alone do not preserve time order.
- Fit preprocessing (e.g., imputation) on training data only, inside each fold.

Cross-Validation: Do Not Trust One Lucky Split

Rotate held-out groups, refit, and summarize the validation scores.



T = train; V = validate; test stays untouched.

- K folds: train K times, validating on a different fold each time.
- Grouped folds keep each delivery run intact.
- The spread measures split sensitivity, not a confidence interval.

Average the held-out scores

$\overline{s} = \frac{1}{K}\sum_{k=1}^{K} s_k$ is the mean held-out score.

K: fold count; k: fold index; s_k: held-out score; $\overline{s}$: mean score. Refit preprocessing within each training fold; leave the test untouched.

A Confusion Matrix Names the Mistakes

On 40 hypothetical segments, count the different mistake types.

	Actual Blocked	Actual Clear
Pred. Blocked	TP $= 9$	FP $= 3$
Pred. Clear	FN $= 6$	TN $= 22$

False positive: unnecessary rerouting.

False negative: the robot enters an unexpectedly blocked route.

High accuracy can hide missed blockages: inspect false negatives and recall.

Four ways to describe performance

$$\text{Accuracy} = \frac{TP + TN}{N} = \frac{31}{40} = 77.5\%$$

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{9}{12} = 75\%$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{9}{15} = 60\%$$

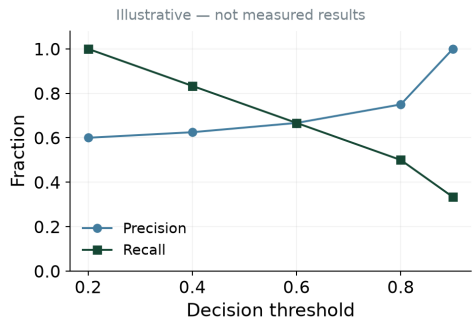
$$F_1 = \frac{2TP}{2TP + FP + FN} = \frac{18}{27} \approx 66.7\%$$

Positive = Blocked. TP/TN: true positives/negatives; FP/FN: false positives/negatives. $N = 40$ cases. F_1: harmonic mean of precision and recall.

A Decision Threshold Turns Scores Into Labels

The same learned score can produce different labels under different risk tolerances.

- Suppose a model assigns blockage score $p = 0.70$ to a corridor.
- Threshold 0.50: predict **blocked**; threshold 0.80: predict **clear**.
- Lowering the threshold cannot reduce recall on a fixed dataset, but adds predicted positives. Precision may rise or fall.



Illustrative precision–recall tradeoff when a threshold changes.

Regression Error: How Far Off Are the Predictions?

Numerical predictions need an error magnitude, not a right/wrong count.

Regression error, if the target is delay in minutes

$$\mathrm{MAE} = \frac{1}{m}\sum_i |\widehat{y}_i - y_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_i (\widehat{y}_i - y_i)^2}.$$

m: evaluated cases; i: case index; y_i: true delay; $\widehat{y}_i$: predicted delay. MAE is mean absolute error; RMSE is root mean squared error. Both use minutes; RMSE emphasizes large misses.

A two-case example

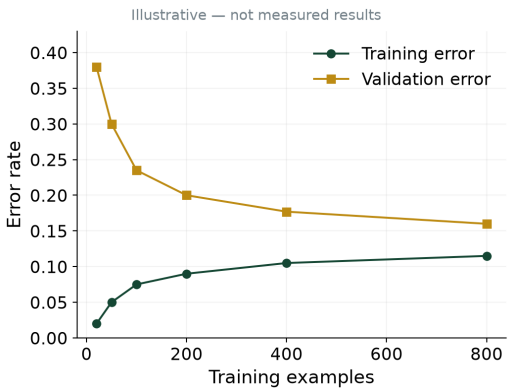
True delays: $[2, 6]$ minutes; predictions: $[3, 3]$ minutes. Absolute errors are $[1, 3]$: MAE $= 2$ minutes and RMSE $= \sqrt{5} \approx 2.24$ minutes.

Key idea

A larger miss influences RMSE more strongly because errors are squared before averaging.

Learning Curves: Does More Training Data Help?

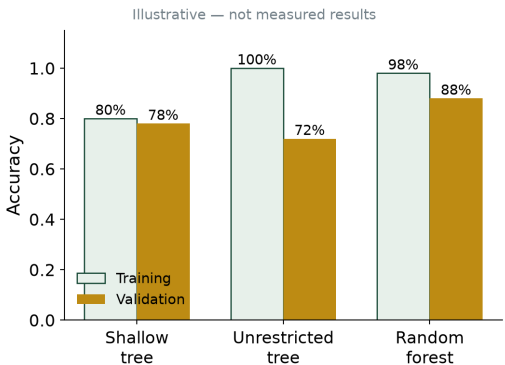
A learning curve shows training and validation performance as sample size grows.



- **Both errors high:** likely underfitting or weak features.
- **Training error low, validation error high:** likely overfitting.
- More representative data may narrow the gap; improvement is not guaranteed.

Choose the Candidate That Works on Validation

The strongest training score need not belong to the most useful model.



- The unrestricted tree gets **100% training accuracy**, but only 72% on validation.
- The random forest gets the best validation score here: **88%**.
- Choose hyperparameters using validation; evaluate the final selected model on test data once.

Key idea

This example illustrates selection, not a guarantee that forests always win.

Why More Data Helps — and What It Cannot Fix

Additional independent experience makes sample-specific accidents less persuasive.

- More *representative* delivery runs show the model what varies and which patterns persist.
- Extra copies of nearly identical frames do not add the same amount of new information.
- More of the *same* data cannot fix missing sensor features, mislabeled outcomes, or a new building unlike the training environments.
- If one validation split gives an unusually good or bad result, grouped cross-validation can expose that instability.

Transition to ensembles

Rather than trust one tree trained on one sample, could we combine many trees that make **different mistakes**?

Ensemble Learning: Combine Imperfect Predictors

One uncertain judgment can become more stable when several useful judgments are combined.

- An **ensemble** uses multiple learned models to produce one prediction.
- For **classification**, models can vote on the label or average their class probabilities.
- For **regression**, models can average their numerical predictions.
- Shared mistakes persist; diverse errors are more likely to cancel.

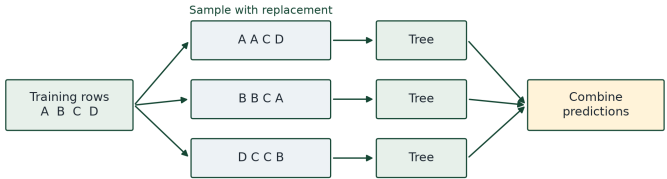
A regression ensemble in compact form

$$\widehat{y}_{\text{ensemble}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B}\widehat{y}_b(\mathbf{x}).$$

$\mathbf{x}$: feature vector for one case; B: model count; b: model index; $\widehat{y}_b(\mathbf{x})$: prediction of model b. Average these to get the ensemble prediction.

Bagging: Bootstrap Aggregation

Train each tree on a newly sampled version of the same training set.



Bootstrap samples train separate trees, whose outputs are combined.

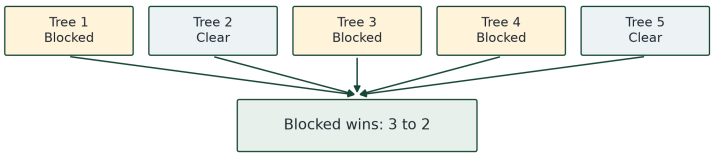
1. Draw N training rows **with replacement**; repetition is allowed.
2. Fit a separate tree to that sample. Repeat for many bootstrap samples.
3. Combine results by majority vote or averaging.

Why the samples really differ

N: original training-set size and draws per sample. The expected distinct fraction is $1 - (1 - 1/N)^N$, about 63% for large N. Related frames should not be mistaken for independent evidence.

Five Trees Vote: A Worked Prediction

The ensemble chooses the class with the most votes — not necessarily a unanimous answer.



Three Blocked votes and two Clear votes.

Classification vote

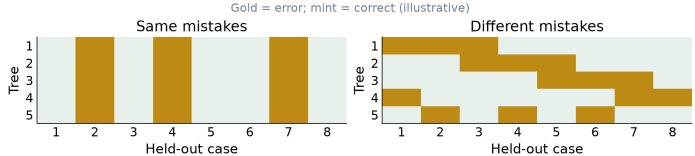
$$\widehat{y}(\mathbf{x}) = \operatorname{mode}\left(\widehat{y}_1(\mathbf{x}), \ldots, \widehat{y}_B(\mathbf{x})\right).$$

$\mathbf{x}$: one case; $\widehat{y}_b$: label from tree b; B: tree count. *Mode* is the most frequent label: Blocked wins 3 to 2 here. Specify a tie rule if needed.

A 3-to-2 vote is not a calibrated 60% probability. In scikit-learn, random forests average tree probabilities rather than hard-vote; shared mistakes can persist.

Why Bagging Similar Trees Helps Less

Correlated errors survive a vote: the trees need reasons to disagree.



- Bootstrap samples vary, but dominant features may repeat. If trees fail on the *same cases*, averaging helps less.

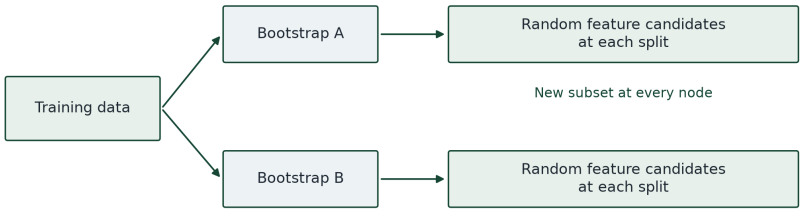
Why correlation matters (optional intuition)

$$\mathrm{Var}(\text{average}) = \sigma^2 \left(\rho + \frac{1-\rho}{B} \right).$$

B: model count; σ^2: variance of each estimate across training samples; ρ: common pairwise correlation. $\rho = 1$: no variance reduction; $\rho = 0$: variance divided by B. This is a variance model, not an accuracy formula.

Random Forests Add a Second Source of Randomness

Random forests randomize the rows used per tree and the candidate features considered at each split.



Randomized observations and random feature subsets encourage diverse trees.

- **Bootstrap rows:** different training samples for different trees.
- **Random feature candidates:** each split searches only a subset of the inputs.
- Each tree chooses its best available split; feature randomness makes trees less alike.

How a Random Forest Trains and Predicts

Many ordinary decision trees are trained; the final output combines their answers.

1. Create a bootstrap sample for each tree.
2. Recursively split with a random subset of candidate features at each node.
3. Let all trees predict the new observation.
4. **Classify:** average class probabilities. **Regress:** average numerical values.

Important hyperparameters

`n_estimators`: tree count.

`max_depth`: depth limit.

`min_samples_leaf`: smallest leaf.

`max_features`: features considered per split.

One common feature-sampling convention

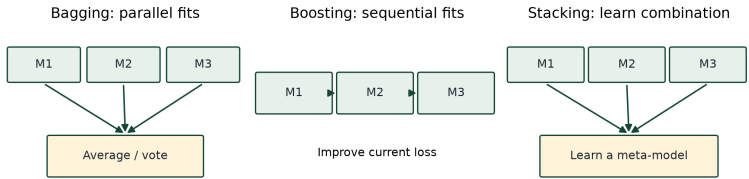
With d input features, `max_features="sqrt"` considers about $\sqrt{d}$ candidate features per split ($\sqrt{\cdot}$: square root). Regression defaults may use all features; the setting is tunable.

Key idea

More trees may stabilize predictions; gains diminish while compute cost grows.

Beyond Random Forests: Other Ensemble Methods

Ensemble learning is a family of approaches, not one algorithm.

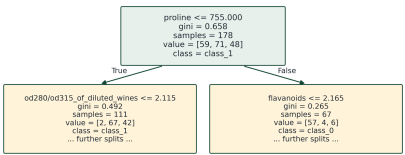


Parallel bagging, sequential boosting, and learned stacking illustrated.

Method	How it combines models
Bagging	Separately fitted bootstrap models combined by averaging/voting; aims to reduce variance.
Random forest	Bagged trees plus random feature candidates to reduce tree correlation.
Boosting	Sequentially adds models to improve errors/loss (AdaBoost, gradient boosting, XGBoost).
Stacking	A meta-model learns from out-of-fold base-model predictions to avoid training leakage.
Voting	Independently trained classifiers combine hard labels or probabilities.

Python Example 1: Explore the Wine Tree

A tree fitted on all data illustrates rules; it does not measure generalization.



Top levels of the depth-4 Wine tree from Lecture 27; fitted on all data.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.tree import (
    DecisionTreeClassifier, plot_tree)
wine = load_wine()
X, y = wine.data, wine.target
clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X, y)
plt.figure(figsize=(14, 7))
plot_tree(clf, max_depth=2, filled=True,
          feature_names=wine.feature_names,
          class_names=wine.target_names, fontsize=8)
plt.show()
```

Wine: Compare on Validation, Then Report on Test

Use the same partitions for both candidates; keep the final test out of selection.

```
# Continue after loading Wine above.
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
X_tr, X_hold, y_tr, y_hold = train_test_split(
    X, y, test_size=.4, stratify=y,
    random_state=42)
X_v, X_test, y_v, y_test = train_test_split(
    X_hold, y_hold, test_size=.5,
    stratify=y_hold, random_state=42)
```

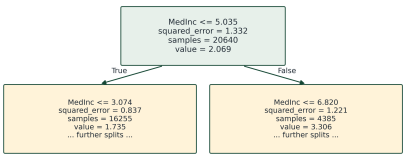
```
models = [
    DecisionTreeClassifier(
        max_depth=4, random_state=42),
    RandomForestClassifier(n_estimators=100,
        max_depth=6, random_state=42)]
for model in models:
    model.fit(X_tr, y_tr)
scores = [m.score(X_v, y_v) for m in models]
best = models[scores.index(max(scores))]
print("Validation accuracy:", scores)
print("Test:", best.score(X_test, y_test))
```

Read the variable names

`tr`: training; `v`: validation; `hold`: temporary held-out pool. `score` is accuracy here; select its maximum. This small independent-sample teaching example uses stratification, not grouped deployment evaluation.

Python Example 2: Explore a Housing Regression Tree

A regression tree predicts a leaf's mean target for the squared-error criterion.



Top levels of the depth-4 California Housing tree from Lecture 27.

```
from sklearn.datasets import (
    fetch_california_housing)
from sklearn.tree import DecisionTreeRegressor
data = fetch_california_housing()
X, y = data.data, data.target
# Structural illustration, not held-out evaluation.
tree = DecisionTreeRegressor(max_depth=4, random_state=42)
tree.fit(X, y)
plt.figure(figsize=(14, 7))
plot_tree(tree, max_depth=2, filled=True,
          feature_names=data.feature_names, fontsize=8)
plt.show()
```

The target is district median house value, in units of \$100,000; rows describe census block groups, not individual houses.

Housing: Select by Validation Error, Then Test Once

For regression, average the tree predictions and prefer lower validation error.

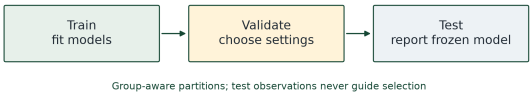
```
# Continue with Housing X, y.
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import root_mean_squared_error
X_tr, X_hold, y_tr, y_hold = train_test_split(
    X, y, test_size=.4, random_state=42)
X_v, X_test, y_v, y_test = train_test_split(
    X_hold, y_hold, test_size=.5, random_state=42)
models = [
    DecisionTreeRegressor(
        max_depth=4, random_state=42),
    RandomForestRegressor(n_estimators=100,
        max_depth=12, random_state=42, n_jobs=-1)]
for model in models:
    model.fit(X_tr, y_tr)
errors = [mean_absolute_error(y_v, m.predict(X_v))
          for m in models]
best = models[errors.index(min(errors))]
pred = best.predict(X_test)
print("Validation MAE:", errors)
print("Test MAE:",
      mean_absolute_error(y_test, pred))
print("Test RMSE:",
      root_mean_squared_error(y_test, pred))
```

What this simple split does and does not test

Both metrics use $100,000 units. Random splits demonstrate the workflow; geographic holdouts are needed to assess transfer to new regions. Code assumes scikit-learn 1.4+ for `root_mean_squared_error`.

Fair Comparisons: Same Inputs, Same Protocol

Changing the model while holding evaluation fixed turns a demonstration into evidence.



Grouped partitions determine when a model may see each observation.

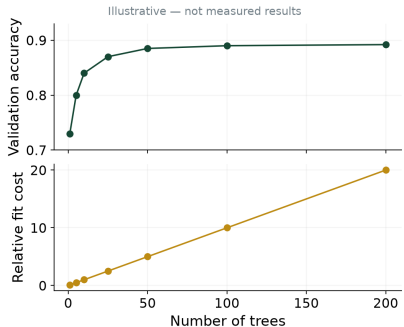
- Fit a **linear baseline**, shallow tree, unrestricted tree, and forest on the same features and train split.
- Tune on validation folds; record depth, accuracy, and runtime.
- Freeze the selected model, then evaluate on test once.

What to vary in the lab

`n_estimators` changes forest size; `max_depth` changes each tree's complexity. Report validation metrics and training-time cost together.

More Trees Are Not a Free Improvement

Forests can be more stable than one deep tree, but they cost computation and cannot repair a broken evaluation.



Illustrative learning-performance saturation and training-time tradeoff.

- Gains can level off while training time continues to grow.
- Highly correlated or systematically biased trees may all make the same mistake.
- A forest is generally less interpretable than one small decision tree.
- No ensemble automatically fixes leakage, missing sensors, or distribution shift.

Synthesis: Better Models Require Better Evidence

1. **Generalization:** evaluate future performance, not just how well the model fits old examples.
2. **Reliable evaluation:** prevent leakage; group correlated observations; use validation to choose settings and test once.
3. **Error diagnosis:** confusion matrices, regression error, and learning curves reveal different kinds of failure.
4. **Ensemble learning:** bagging combines bootstrap-fitted predictors; random forests add feature randomness to lower correlation.
5. **Limits:** averaging can reduce variance, not recover information that was never measured.

Key idea

When would five identical trees fail together, and why would five diverse trees do better?